PRODUCT REQUIREMENTS DOCUMENT

QueryTrust AI: Governed Natural-Language Analytics Workbench

Trustworthy NL-to-SQL, Multi-Layer AST Safety, Telemetry Outlier Detection, and Regression Benchmark Suite

Independent student capstone project | Version 1.0 | 14 October 2026

---

1. Problem

Enterprise business users and operations engineers frequently need answers from relational databases (e.g., asset counts, warranty schedules, incident trends, regional hardware health) but lack SQL proficiency, relational schema knowledge, or statistical tools. While modern Large Language Models (LLMs) can translate natural language into SQL, naive text-to-SQL implementations introduce catastrophic risks: 1. **Security Vulnerabilities:** Prompt-based instructions fail as security boundaries. Malicious or hallucinated queries can execute destructive DDL/DML (`DROP` , `DELETE` , `UPDATE`), access sensitive system catalogs (`pg_catalog` , `information_schema`), perform out-of-band data exfiltration (`dblink` , `pg_read_file`), or trigger Denial-of-Service (`pg_sleep` , unbounded cross-joins). 2. **Semantic Hallucinations & Silent Logic Errors:** LLMs invent nonexistent tables/columns, misinterpret undefined business concepts (e.g., fabricating arbitrary logic for "high-risk" or "unhealthy"), and assume financial metrics (revenue, profit, churn) exist within purely operational hardware schemas. 3. **Execution Opacity & Runaway Queries:** Standard chat interfaces execute generated queries immediately and blindly, without letting users preview the SQL, inspect AST validation findings, or understand assumptions. Runaway queries without row limits or execution timeouts lock database connections. 4. **Lack of Verifiable Measurement:** Teams deploy generative analytics without regression test suites or baseline-versus-guarded comparisons, relying on cherry-picked demos and uncalibrated claims.

Project Scope & Academic Boundary:
QueryTrust AI builds a local-first, full-stack, governed natural-language analytics workbench. It demonstrates that combining schema-aware semantic context, formal AST parsing via SQLGlot, multi-layer policy enforcement, dual database roles with strict `SELECT` -only privileges, and a repeatable regression benchmark delivers trustworthy analytics. This student project runs self-contained without proprietary corporate infrastructure, customer datasets, or paid cloud services. Real Cisco or client data, logos, or commercial affiliations are strictly excluded; all data is deterministically synthesized for educational evaluation.

---

2. Product Goal

Build a production-grade, local full-stack web application where an operator, analyst, or administrator can:

1. **Explore Hardware Fleet Intelligence:** Inspect KPI summaries, regional distributions, warranty expiration horizons, and open service incident trends over synthetic networking assets.
2. **Ask Natural-Language Data Questions:** Formulate questions in plain English (e.g., *"Show active routers in the North region with warranties expiring next month"*).
3. **Preview & Inspect Governed SQL Before Execution:** Review the candidate SQL query, interpreted intent, assumptions, and a multi-layer Abstract Syntax Tree (AST) safety checklist before any database interaction occurs.
4. **Experience Deterministic Disambiguation:** Receive structured clarification options when requesting undefined business terms (e.g., "high-risk devices") and clear out-of-domain rejections for unsupported concepts (e.g., revenue, churn) without hallucinated SQL.
5. **Execute Under Dual-Layer Sandboxing:** Run server-stored, validated query previews via a dedicated PostgreSQL read-only database role (`analytics_reader`) with strict row clamping and statement timeouts.
6. **Profile Data Quality:** Run deterministic data profiling across asset tables to detect missing fields, invalid date sequences, inconsistent categories, duplicate serial numbers, and telemetry range violations.

7. **Monitor Telemetry Outliers:** Scan device telemetry using a genuine scikit-learn `IsolationForest` model to flag multivariate operational anomalies, explicitly presented as statistical outliers rather than device failure probabilities.
8. **Evaluate & Benchmark Reliability:** Execute a 30+ case automated evaluation suite comparing a raw LLM baseline against the guarded QueryTrust pipeline, measuring execution correctness, safe rejection rate, clarification accuracy, and latency.

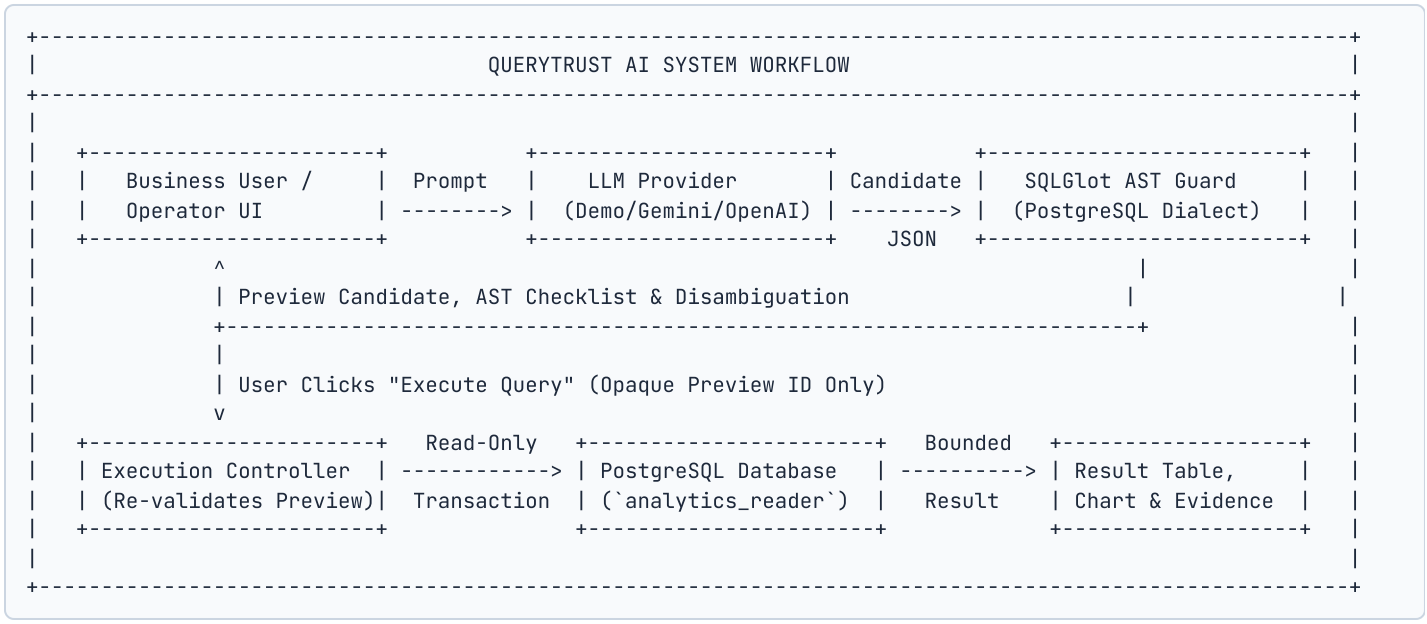


Figure 1. The natural-language query path enforces complete separation between preview translation and governed execution.

System Invariants & Truthfulness Rules

- **Natural-Language Path is Read-Only for All Roles:** Even System Administrators cannot execute data-modifying SQL through natural language.
- **Never Fabricate Execution or Model Metrics:** If no live LLM key is configured, the application operates in an honest, visibly badged `Demo Mode`. Accuracy metrics, anomaly scores, and test passes are never hard-coded or faked.
- **Preview ID Execution Only:** The execution API accepts only an opaque, server-stored preview ID (`query_run_id`). The browser cannot submit raw SQL for execution.
- **IsolationForest is NOT Failure Prediction:** Telemetry anomalies are labeled strictly as statistical multivariate outliers; they are never represented as probabilities of hardware failure.

---

3. Users and Core Stories

Role	Core Purpose	Access Boundary
Viewer	Operational staff reviewing hardware inventory and asking read-only questions	Reads dashboard, devices, sites; asks NL queries; views own history. Cannot run profiling, anomaly scans, admin tasks, or write to database.
Analyst	Technical lead diagnosing fleet health, data hygiene, and telemetry	All Viewer capabilities, plus initiates Data Quality profiling, runs Telemetry Anomaly scans, and submits query feedback.
Administrator	System owner auditing security, managing users, and verifying reliability	All Analyst capabilities, plus user management, security audit log access, and execution of the Evaluation Lab regression benchmark.
Assessor / Reviewer	Academic or technical evaluator validating project requirements	Evaluates clean-clone setup, automated test suite, security injection blocks, demo mode, and live execution.

Core User Stories

1. **As a Viewer**, I want to ask questions about our network devices in plain English so that I can find operational answers without knowing SQL syntax.
2. **As a Viewer**, I want to see the interpreted intent and generated SQL before execution so that I can verify the system understood my question correctly.
3. **As an Analyst**, I want the system to ask for clarification when I use ambiguous terms like "high-risk" so that business decisions are not based on hidden assumptions.
4. **As an Analyst**, I want the system to decline questions about revenue or churn immediately so that I know the boundary of the operational inventory dataset.
5. **As an Analyst**, I want to profile the device database for missing warranties, bad dates, and duplicate serials so that dirty data can be identified and remediated.
6. **As an Analyst**, I want to run an anomaly scan over telemetry readings so that devices exhibiting unusual temperature, latency, or packet loss can be investigated.
7. **As an Administrator**, I want all natural-language queries executed through a restricted, `SELECT`-only database role so that our core tables can never be altered or dropped.
8. **As an Administrator**, I want to run an automated regression benchmark so that I can measure whether prompt or code changes improve accuracy or introduce regressions.
9. **As an Assessor**, I want to start the application with a single Docker Compose command and run tests in offline demo mode without needing paid API keys.

---

4. Required Implementation Stack

Layer	Required Choice	Purpose / Architectural Rationale
Frontend Framework	React 18/19, TypeScript, Vite	Fast, typed, reactive single-page application matching modern web standards.
Frontend Styling	Custom CSS Design System / Modern Vanilla CSS tokens	Ultra-crisp, responsive, enterprise-grade dark/light theme without bulky framework bloat.
Icons & Charts	Lucide React, SVG visualization	Scalable, clean iconography and responsive tabular/chart data rendering.
Backend Framework	Python 3.12 / 3.13, FastAPI, Uvicorn	Async, high-performance REST API with automatic OpenAPI documentation and strict typing.
Data Validation	Pydantic v2	Robust, type-safe serialization, deserialization, and schema enforcement for all endpoints.
Database	PostgreSQL 16+	Production-grade relational database supporting foreign keys, transactional integrity, and granular RBAC.
ORM & Migrations	SQLAlchemy 2.x, Alembic	Declarative database models, connection pooling, and version-controlled schema migrations.
Database Security	Dual PostgreSQL Roles (`app_user` , `analytics_reader`)	Hardened defense-in-depth: `analytics_reader` possesses zero `INSERT` / `UPDATE` / `DELETE` / `DDL` privileges.
SQL AST Guard	SQLGlot (PostgreSQL dialect)	Deep AST inspection, single-statement verification, clause inspection, table allowlisting, and row limit clamping.
ML Inference	scikit-learn `IsolationForest` , NumPy, Pandas	Unsupervised outlier detection on device telemetry, with serialized model artifacts and metadata tracking.
Authentication	Passlib (Bcrypt), Python-JOSE (JWT)	Secure password hashing, signed short-lived access tokens, and backend dependency-based RBAC.
Testing	pytest, pytest-asyncio, HTTPX	Comprehensive automated unit, integration, and security test suites with zero external network dependencies.
Containerization	Docker, Docker Compose	Reproducible multi-service deployment (PostgreSQL, backend, frontend) with volume persistence and health checks.

Local Prerequisites

- **Python:** 3.12+ or 3.13+ (tested with 3.13.9)
- **Node.js & npm:** Node.js 20+ (tested with v24.14.1), npm 10+ (tested with 11.11.0)
- **Database:** Local PostgreSQL 16+ or Docker Compose
- **Git:** Standard git client

Required Repository Structure

```
querytrust-ai/
├── docker-compose.yml              # Multi-container local orchestration (Postgres, Backend, Frontend)
├── .env.example                    # Documented template for all required runtime environment variables
├── .gitignore                      # Excludes .env, pycache, dist, and local artifacts
├── AGENTS.md                       # Persistent engineering invariants and rules
├── PROJECT_SPEC.md                 # Master project specification
├── ANTIGRAVITY_MASTER_PROMPT.md    # Multi-milestone execution plan
├── README.md                       # Comprehensive setup, run, and evaluation documentation
├── docs/
│   ├── PRD.md                      # This official Product Requirements Document
│   ├── BUILD_STATUS.md             # Verifiable milestone completion record and test logs
│   ├── ARCHITECTURE.md             # Detailed component diagrams and data flow contracts
│   ├── SECURITY.md                 # Threat model, AST validation rules, and RBAC matrix
│   └── EVALUATION_PLAN.md          # Benchmark test cases, metrics, and regression methodology
├── backend/
│   ├── pyproject.toml              # Python project metadata, dependencies, and pytest configuration
│   ├── alembic.ini                 # Alembic migration configuration
│   ├── alembic/                    # Schema migration scripts
│   ├── app/
│   │   ├── main.py                 # FastAPI application factory, middleware, and router registration
│   │   ├── core/                   # Security, settings, logging, SQL validator, semantic catalog, LLM provider
│   │   │   ├── config.py           # Typed environment settings (Pydantic BaseSettings)
│   │   │   ├── security.py         # Password hashing, JWT token creation, and auth dependencies
│   │   │   ├── sql_validator.py    # SQLGlot AST safety validator and row-clamp engine
│   │   │   ├── semantic_catalog.py # Business metrics, table definitions, approved joins, ambiguity rules
│   │   │   └── llm_provider.py     # BaseLLMProvider, DemoLLMProvider, GeminiLLMProvider, OpenAICompatible
│   │   ├── db/                     # Database sessions, base, and models
│   │   │   ├── session.py          # Dual engine setup: app_user (read/write) & analytics_reader (read-only)
│   │   │   └── models.py           # SQLAlchemy declarative models (users, devices, sites, telemetry, etc.)
│   │   ├── schemas/                # Pydantic request and response schemas
│   │   │   ├── auth.py             # Login, Token, UserResponse schemas
│   │   │   ├── analytics.py        # PreviewRequest, PreviewResponse, ExecuteRequest, ExecuteResponse
│   │   │   ├── chat.py             # ChatSessionCreate, ChatMessageResponse schemas
│   │   │   ├── data_quality.py     # DataQualityRunResponse, Finding schemas
│   │   │   └── anomaly.py          # TelemetryScanRequest, AnomalyResult schemas
│   │   ├── services/               # Core business and algorithmic logic
│   │   │   ├── analytics_service.py# Preview generation, AST validation, execution, and history
│   │   │   ├── data_quality.py     # Deterministic data profiling rules engine
│   │   │   ├── anomaly_service.py  # IsolationForest fitting, serialization, and telemetry scoring
│   │   │   └── evaluation_service.py# 30+ case benchmark runner and regression comparator
│   │   ├── api/v1/                 # Versioned REST API endpoints
│   │   │   ├── auth.py             # /api/v1/auth (login, me)
│   │   │   ├── dashboard.py        # /api/v1/dashboard (summary KPI cards and distributions)
│   │   │   ├── devices.py          # /api/v1/devices (paginated catalog, search, filters)
│   │   │   ├── analytics.py        # /api/v1/analytics (preview, execute, feedback, history)
│   │   │   ├── chat.py             # /api/v1/chat/sessions (conversation management and message retrieval)
│   │   │   ├── data_quality.py     # /api/v1/data-quality (run, summary)
│   │   │   ├── anomalies.py        # /api/v1/anomalies (scan, summary)
│   │   │   ├── evaluations.py      # /api/v1/evaluations (run, results)
│   │   │   └── admin.py            # /api/v1/admin (users, audit logs)
│   │   └── seed/                   # Deterministic synthetic data generation
│   │       └── seed_database.py    # Generates 5,000 devices, 40 sites, service events, and telemetry
│   └── tests/                      # Automated test suite (62+ tests)
│       ├── conftest.py             # Test database fixtures and mock clients
│       ├── test_auth.py            # Authentication and RBAC tests
│       ├── test_sql_validator.py   # SQLGlot AST validation and security injection tests
│       ├── test_llm_provider.py    # LLM provider and demo rule engine tests
│       ├── test_analytics_preview.py # Preview generation and clarification tests
│       ├── test_analytics_execute.py # Read-only execution and preview ID enforcement tests
│       ├── test_chat_sessions.py   # Session ownership isolation tests
│       ├── test_data_quality.py    # Data profiling rule verification tests
│       ├── test_anomalies.py       # IsolationForest inference and metadata tests
```

```
│       └── test_database_security.py# Verification that analytics_reader cannot perform write operations
├── frontend/
│   ├── package.json                # React, Vite, TypeScript dependencies
│   ├── tsconfig.json               # Strict TypeScript compiler options
│   ├── vite.config.ts              # Vite dev server and proxy configuration
│   ├── index.html                  # HTML entry point with modern typography
│   └── src/
│       ├── main.tsx                # React DOM mount point
│       ├── App.tsx                 # Root router, global layout, navigation shell, user context
│       ├── index.css               # Comprehensive design system tokens, typography, and component styling
│       ├── api/
│       │   └── client.ts           # Strictly-typed HTTP client wrapping all /api/v1 endpoints
│       └── components/
│           ├── OverviewDashboard.tsx # KPI summary cards, warranty horizons, incident distributions
│           ├── DeviceCatalog.tsx   # Paginated device list, status filters, search, site mapping
│           ├── AskDataView.tsx     # Natural-language workbench, AST checklist, SQL preview, execution
│           ├── DataQualityView.tsx # Data hygiene findings table, severity filters, remediation notes
│           ├── AnomalyMonitorView.tsx # Telemetry anomaly scatter, outlier ranking, model metadata
│           ├── EvaluationLabView.tsx # Benchmark test runner, regression comparator, pass-rate metrics
│           └── AdminView.tsx       # User management, role transitions, security audit logs
└── scripts/
    ├── seed.ps1 / seed.sh          # Database seeding helper
    └── run_tests.ps1 / run_tests.sh# Test runner script
```

---

5. AI Access, Semantic Catalog, and Local Mode

5.1 Pluggable LLM Provider Architecture

The application defines a strict `BaseLLMProvider` interface to decouple business logic from external model vendors: 1. `DemoLLMProvider` **(Default / Local Mode):**
- Operates 100% offline without external network calls or API credentials. - Powered by a deterministic semantic parser covering fleet aggregations, regional filtering, warranty horizons, incident triage, ambiguity disambiguation, out-of-domain detection, and injection testing. - The UI displays an honest badge: `Mode: Demo (Deterministic Rule Engine)`. It never claims a live LLM was invoked. 2. `GeminiLLMProvider` :
- Integrates Google Gemini API using `GEMINI_API_KEY`. - Uses structured outputs (`response_mime_type="application/json"`) with single-turn repair retries on malformed syntax. 3. `OpenAICompatibleLLMProvider` :
- Connects to OpenAI or any compatible local endpoint (e.g., vLLM, Ollama, Groq) via `OPENAI_API_KEY` and `OPENAI_BASE_URL` .

Security Rule: API keys are stored exclusively in backend environment variables (`.env`). Keys are never sent to the browser, logged in application outputs, or written into audit artifacts.

5.2 Central Semantic Catalog

Located in `app/core/semantic_catalog.py` , the semantic catalog serves as the single source of truth for database schema definitions, joins, and business terms: - **Allowed Tables:** Strictly `devices` , `sites` , `service_events` . - **Approved Joins:** - `devices.site_id → sites.id` - `service_events.device_id → devices.id` - **Formal Business Metric Definitions:** - *Device Count:* `COUNT(DISTINCT devices.id)` - *Active Device:* `LOWER(TRIM(devices.status)) = 'active'` - *Open Service Event:* `LOWER(TRIM(service_events.status)) NOT IN ('resolved', 'closed', 'cancelled')` - *Warranty Expiring in N Days:* `devices.warranty_expiry ⩾ '2026-10-14' AND devices.warranty_expiry ⩽ ('2026-10-14'::date + INTERVAL 'N days')` (uses fixed reference date for reproducible evaluation). - **Ambiguity Disambiguation Rules:** - The term *"high-risk"* has no intrinsic definition. The catalog mandates returning `intent_status: clarification_needed` with explicit choices: 1. Imminent Warranty Expiry (`< 30 days`) 2. Open Critical Service Incidents (`severity = 'critical'`) 3. Offline / Degraded Status - **Explicit Unsupported Domain Rejections:** - Concepts such as `revenue` , `profit` , `cost` , `churn` , and `salary` are defined as unsupported. The catalog mandates returning `intent_status: unsupported` with an explanation that the database is an operational networking inventory, preventing hallucinations.

6. Student Workflow and Screens

The workbench follows an enterprise-grade operator workflow across 7 functional screens:

Step	Screen	Required Behavior
1	**Authentication & Shell**	Sign in with email/password; development mode provides quick-switch demo pills (`Admin` , `Analyst` , `Viewer`). Top banner displays current user role, live LLM mode, and synthetic data disclosure.
2	**Overview Dashboard**	Displays 5 live KPI cards (Total Devices, Active Fleet, 30-Day Expiring Warranties, Open Incidents, Quality Issues). Renders device status distribution, warranty expiry trend, and open incidents by severity from live PostgreSQL data.
3	**Device Catalog**	Paginated inventory table with search, device type filters (`switch` , `router` , `access_point` , `firewall`), status filters, and site location mappings.
4	**Ask Data (Workbench)**	Multi-turn chat interface with left conversation session drawer. Starter prompt chips for fast testing. Displays interpreted intent, clarification buttons, unsupported domain notices, SQL preview, AST checklist, and "Execute Candidate Query" action.
5	**Execution & Visuals**	Renders query results in a responsive data grid with row count and duration badges. Automatically recommends and renders charts (bar, pie, metric card) for aggregate queries. Provides "Helpful / Not Helpful" feedback widget.
6	**Data Quality**	Runs deterministic quality profiling across assets. Displays summary hygiene score, categorized findings table (Missing Fields, Invalid Dates, Category Inconsistencies, Duplicate Serials), severity badges, and remediation suggestions.
7	**Anomaly Monitor**	Runs `IsolationForest` telemetry scan over 30,000+ data points. Renders device telemetry scatter plot, top-anomalies data grid (CPU, Memory, Temp, Packet Loss, Latency), model version, and synthetic outlier disclaimer.
8	**Evaluation Lab (Admin)**	Admin-only suite running 30+ benchmark cases. Displays execution correctness %, safe rejection rate %, multi-turn accuracy %, latency percentiles, and test-by-test failure diagnostics.
9	**Administration (Admin)**	Admin-only user management table, active status toggles, role transitions, and immutable security audit log viewer.

Visual Direction and UI Quality Bar

- **Design System:** Deep slate/navy shell (`#0f172a` , `#1e293b`), crisp neutral white/gray workspace surfaces, restrained indigo (`#6366f1`) and teal (`#14b8a6`) accents, and emerald/amber/rose semantic status badges.
- **Typography:** Modern clean sans-serif (Inter / System UI) with strict hierarchical sizing, monospace code blocks (`Fira Code` , `JetBrains Mono`), and high-contrast readability.
- **Responsiveness & Accessibility:** Fully responsive design tested at 390px (mobile), 768px (tablet), and 1440px (desktop). Keyboard navigation support (`Enter` to submit query, `Shift+Enter` for newline, accessible modal dialogs, visible focus outlines).
- **State Handling:** Real skeleton loaders, non-empty initial states with starter chips, actionable error banners with diagnostic details, and zero silent failures or placeholder buttons.

7. Core Data Model and Synthetic Dataset Contract

7.1 Database Schema Specifications

```
-- 1. Users & RBAC
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) UNIQUE NOT NULL,
    display_name VARCHAR(100) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(20) NOT NULL CHECK (role IN ('admin', 'analyst', 'viewer')),
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 2. Sites / Locations
CREATE TABLE sites (
    id SERIAL PRIMARY KEY,
    site_code VARCHAR(20) UNIQUE NOT NULL,
    site_name VARCHAR(100) NOT NULL,
    city VARCHAR(100) NOT NULL,
    region VARCHAR(50) NOT NULL
);

-- 3. Devices (Hardware Inventory)
CREATE TABLE devices (
    id SERIAL PRIMARY KEY,
    asset_tag VARCHAR(50) UNIQUE NOT NULL,
    device_type VARCHAR(50) NOT NULL,
    vendor VARCHAR(50) NOT NULL,
    model VARCHAR(100) NOT NULL,
    firmware_version VARCHAR(50),
    site_id INTEGER NOT NULL REFERENCES sites(id) ON DELETE RESTRICT,
    status VARCHAR(30) NOT NULL,
    installed_at DATE NOT NULL,
    warranty_expiry DATE,
    serial_number VARCHAR(100),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 4. Service Events (Operational Incidents)
CREATE TABLE service_events (
    id SERIAL PRIMARY KEY,
    device_id INTEGER NOT NULL REFERENCES devices(id) ON DELETE CASCADE,
    event_type VARCHAR(50) NOT NULL,
    severity VARCHAR(20) NOT NULL CHECK (severity IN ('low', 'medium', 'high', 'critical')),
    status VARCHAR(30) NOT NULL,
    opened_at TIMESTAMPTZ NOT NULL,
    closed_at TIMESTAMPTZ,
    description TEXT NOT NULL
);

-- 5. Telemetry Readings (Operational Metrics)
CREATE TABLE telemetry_readings (
    id SERIAL PRIMARY KEY,
    device_id INTEGER NOT NULL REFERENCES devices(id) ON DELETE CASCADE,
    recorded_at TIMESTAMPTZ NOT NULL,
    cpu_percent NUMERIC(5,2) NOT NULL,
    memory_percent NUMERIC(5,2) NOT NULL,
    temperature_c NUMERIC(5,2) NOT NULL,
    packet_loss_percent NUMERIC(5,2) NOT NULL,
```

```
    latency_ms NUMERIC(7,2) NOT NULL,
    known_injected_outlier BOOLEAN NOT NULL DEFAULT FALSE
);

-- 6. Chat Sessions & Messages
CREATE TABLE chat_sessions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    title VARCHAR(200) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE chat_messages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id UUID NOT NULL REFERENCES chat_sessions(id) ON DELETE CASCADE,
    role VARCHAR(20) NOT NULL CHECK (role IN ('user', 'assistant', 'system')),
    content TEXT NOT NULL,
    metadata_json JSONB,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 7. Query Runs & Auditing
CREATE TABLE query_runs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    session_id UUID REFERENCES chat_sessions(id) ON DELETE SET NULL,
    natural_query TEXT NOT NULL,
    interpreted_intent TEXT,
    sql_query TEXT,
    status VARCHAR(30) NOT NULL, -- previewed, executed, blocked, clarification_needed, unsupported, failed
    validation_findings JSONB,
    provider_name VARCHAR(50) NOT NULL,
    prompt_version VARCHAR(20) NOT NULL,
    row_count INTEGER,
    execution_duration_ms INTEGER,
    error_message TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    executed_at TIMESTAMPTZ
);

-- 8. Query Feedback
CREATE TABLE query_feedback (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    query_run_id UUID NOT NULL REFERENCES query_runs(id) ON DELETE CASCADE,
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    is_helpful BOOLEAN NOT NULL,
    comment TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 9. Data Quality & Anomaly Model Metadata
CREATE TABLE data_quality_runs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id),
    total_records_scanned INTEGER NOT NULL,
    findings_count INTEGER NOT NULL,
    summary_metrics JSONB NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE evaluation_runs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id),
```

```
    benchmark_version VARCHAR(50) NOT NULL,
    provider_name VARCHAR(50) NOT NULL,
    total_cases INTEGER NOT NULL,
    passed_cases INTEGER NOT NULL,
    failed_cases INTEGER NOT NULL,
    pass_rate_percent NUMERIC(5,2) NOT NULL,
    metrics_by_category JSONB NOT NULL,
    failure_details JSONB,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

7.2 Deterministic Synthetic Data Generator

Executed via `python -m app.seed.seed_database`, the generator populates realistic but strictly synthetic records using a fixed random seed (`seed=42`): - **Scale:** 50 sites across 5 regions, 5,000 devices, 8,500 service events, and 35,000 telemetry records. - **Injected Controlled Data Hygiene Issues:** - 120 devices with null `warranty_expiry`. - 45 devices with `warranty_expiry < installed_at` (invalid chronological sequence). - 30 duplicate serial numbers. - Inconsistent casing and whitespace in status (`Active`, `decommissioned`, `MAINTENANCE`). - Inconsistent device types (`switch`, `Switch`, `cisco_switch`). - 350 telemetry records with extreme values (CPU > 98%, Temp > 92°C, Packet Loss > 25%).

---

8. SQL AST Safety and Read-Only Execution Contract

QueryTrust AI enforces strict defense-in-depth where prompt guidelines are backed by multi-stage deterministic code gates and database-level permission restrictions.



Figure 2. Complete multi-stage AST validation and database-level read-only containment.

---

9. Backend API Contract

All business endpoints are strictly versioned under `/api/v1` and documented automatically via FastAPI OpenAPI (`/docs`).

Method	Endpoint	Required Role	Description
`GET`	`/health`	Public	Service liveness probe (returns `{ "status": "ok" }`).
`GET`	`/ready`	Public	Database readiness probe (verifies database connectivity).
`POST`	`/api/v1/auth/login`	Public	OAuth2 password flow; issues signed JWT bearer access token.
`GET`	`/api/v1/auth/me`	Authenticated	Returns current authenticated user profile and role.
`GET`	`/api/v1/dashboard/summary`	Authenticated	Returns live counts, warranty expiration buckets, and incident distributions.
`GET`	`/api/v1/devices`	Authenticated	Paginated device listing with search, type, status, and site filters.
`GET`	`/api/v1/sites`	Authenticated	Returns site codes, names, cities, and regions.
`POST`	`/api/v1/chat/sessions`	Authenticated	Creates a new user conversation session.
`GET`	`/api/v1/chat/sessions`	Authenticated	Lists conversation sessions belonging to the current user.
`GET`	`/api/v1/chat/sessions/{id}/messages`	Authenticated	Retrieves conversation message history (strictly owner-scoped; 403 on isolation breach).
`DELETE`	`/api/v1/chat/sessions/{id}`	Authenticated	Deletes a conversation session and cascades to messages.
`POST`	`/api/v1/analytics/preview`	Authenticated	Translates natural query, runs AST validation, persists candidate, returns preview ID. **Never executes SQL.**
`POST`	`/api/v1/analytics/execute`	Authenticated	Executes validated preview by ID using `analytics_reader` role. Enforces row clamping and timeout.
`POST`	`/api/v1/analytics/feedback`	Authenticated	Records user helpfulness rating and explanation for a query run.
`GET`	`/api/v1/analytics/history`	Authenticated	Returns user query history; Admin may view global metadata.
`GET`	`/api/v1/data-quality/summary`	Analyst / Admin	Retrieves findings from the most recent data quality run.
`POST`	`/api/v1/data-quality/run`	Analyst / Admin	Triggers deterministic data profiling across devices and telemetry.
`GET`	`/api/v1/anomalies/summary`	Analyst / Admin	Retrieves latest telemetry anomaly results and model metadata.
`POST`	`/api/v1/anomalies/scan`	Analyst / Admin	Runs `IsolationForest` inference on recent telemetry.
`GET`	`/api/v1/evaluations`	Admin	Lists historical evaluation benchmark runs.
`POST`	`/api/v1/evaluations/run`	Admin	Executes the 30+ case regression benchmark and returns metrics.
`GET`	`/api/v1/admin/users`	Admin	Lists all user accounts with roles and active statuses.
`PATCH`	`/api/v1/admin/users/{id}`	Admin	Modifies user role or active status.
`GET`	`/api/v1/admin/audit-logs`	Admin	Retrieves security logs and query execution audit records.

Sample Preview Request & Response (`POST /api/v1/analytics/preview`)

Request:

```
{
  "query": "Show active routers in the North region",
  "session_id": "7b8d689c-3642-4919-8669-e07e32402120"
}
```

Response (HTTP 200 OK):

```
{
  "query_run_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "intent_status": "ready",
  "interpreted_question": "List all active devices of type 'router' located at sites in the North region.",
  "clarification_question": null,
  "sql_preview": "SELECT d.asset_tag, d.model, d.status, s.site_name, s.region FROM devices AS d JOIN sites AS s
  "validation": {
    "passed": true,
    "checks": [
      "single_statement_ast",
      "select_only_ast",
      "table_allowlist",
      "row_clamping_active"
    ],
    "warnings": []
  },
  "assumptions": [
    "Status matched case-insensitively using semantic rule LOWER(TRIM(status)) = 'active'",
    "Row count automatically clamped to maximum of 200 rows"
  ],
  "can_execute": true,
  "provider_mode": "demo"
}
```

Sample Execute Request & Response (`POST /api/v1/analytics/execute`)

Request:

```
{
  "query_run_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d"
}
```

Response (HTTP 200 OK):

```
{
  "query_run_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "status": "executed",
  "columns": ["asset_tag", "model", "status", "site_name", "region"],
  "rows": [
    ["RTR-N-0102", "ASR 1001-X", "Active", "Chicago Core DC", "North"],
    ["RTR-N-0205", "Catalyst 8300", "active", "Minneapolis Branch", "North"]
  ],
  "row_count": 2,
  "execution_duration_ms": 14,
  "executed_sql": "SELECT d.asset_tag, d.model, d.status, s.site_name, s.region FROM devices AS d JOIN sites AS
  "chart_recommendation": {
    "type": "table",
    "x_axis": null,
    "y_axis": null
  },
  "explanation": "Found 2 active routers operating across North region sites."
}
```


Standard Error Envelope

All error responses adhere to a consistent contract across all endpoints:


```
{
  "detail": {
    "error_code": "FORBIDDEN_OPERATION",
    "message": "AST validation failed: Prohibited statement type 'Drop' detected. Only single read-only SELECT s
    "request_id": "req-98fbc102"
  }
}
```


---

10. Validation, Failure Handling, and Safety Invariants

1. **Revalidate on Execution:** When `POST /api/v1/analytics/execute` is invoked with a `query_run_id`, the backend retrieves the persisted SQL and runs it through the SQLGlot AST validator a second time before submitting it to the read-only database session.
2. **Session Ownership Isolation:** Conversation sessions (`/api/v1/chat/sessions`) and query histories are strictly partitioned by `user_id`. Attempting to access another user's session yields HTTP 403 Forbidden.
3. **No Execution of Clarifications or Blocked Queries:** If `intent_status` is `clarification_needed`, `unsupported`, or `blocked`, `can_execute` is set to `false`. The `/execute` endpoint rejects any attempt to execute these records with HTTP 400 Bad Request.
4. **Transient Outage Resilience:** If an external LLM provider fails (HTTP 429, 503, or timeout), the backend catches the error, retries once with a backoff, and falls back gracefully to a structured error or demo mode. The user's input and session state are never lost.
5. **Data Minimization:** Query execution returns only the columns and rows required to answer the prompt. Application logs record query metadata, duration, and error codes, but never log raw database passwords, tokens, or result rows.

---

11. Demo and Acceptance Criteria

The system must successfully complete this verification scenario evaluated by an independent assessor:

Evaluation Scenario: "Regional Hardware Audit & Risk Triage"

1. **Startup & Seeding:** The assessor starts the backend and frontend using `docker-compose up` or the local run commands from the README. The seed command deterministically loads 50 sites, 5,000 devices, service events, and telemetry.

2. **Authentication:** Assessor signs in as `viewer@querytrust.local` via the quick-switch demo login. Verified: viewer cannot see Admin tabs (Evaluation Lab, Administration).
3. **Fleet Overview:** Dashboard displays live counts matching PostgreSQL (`devices: 5000` , `sites: 50`). Warranty expiration bucket charts render from live database queries.
4. **Governed NL Query (Happy Path):**
5. Assessor navigates to **Ask Data** and enters: *"Active devices by region"*.
6. System displays interpreted question, green `Preview Validated` status, the PostgreSQL SELECT query, and 4 passing AST checklist items.
7. Assessor clicks **Execute Candidate Query**. Result table displays counts grouped by region in under 50ms.
8. **Ambiguity Disambiguation Workflow:**
9. Assessor submits: *"Show high-risk devices in New York"*.
10. System immediately displays an amber `Clarification Needed` card explaining that "high-risk" has multiple operational definitions, with 3 selectable options:
    - Option 1: Imminent Warranty Expiry (`< 30 days`)
    - Option 2: Open Critical Incidents
    - Option 3: Offline / Degraded Status
11. Verified: The Execute button is suppressed.
12. **Out-of-Domain Concept Rejection:**
13. Assessor submits: *"What is the quarterly revenue and customer churn?"*
14. System immediately displays a blue `Unsupported Domain` card explaining that revenue and churn do not exist in this networking asset schema. Zero SQL is generated.
15. **Security Injection Defense:**
16. Assessor submits: `SELECT * FROM devices; DROP TABLE sites; --`
17. System triggers a red `AST Security Violations Detected` alert identifying `Prohibited statement type 'Drop'` . Execution is permanently blocked.
18. **Data Quality Profiling:**
19. Assessor switches user to `analyst@querytrust.local` , opens **Data Quality**, and clicks **Run Profile**.
20. Profile completes and displays 5 categorized findings: missing warranties (120 rows), invalid install vs expiry dates (45 rows), duplicate serial numbers (30 rows), and category inconsistencies.
21. **Telemetry Anomaly Scan:**
22. Assessor opens **Anomaly Monitor** and clicks **Run Telemetry Scan**.
23. scikit-learn `IsolationForest` scores telemetry readings, displays the anomaly scatter distribution, and lists top outliers with a clear notice: *"Outlier signals only; not a device failure probability."*
24. **Automated Regression Benchmark (Admin):**
    - Assessor switches to `admin@querytrust.local` , opens **Evaluation Lab**, and clicks **Run Evaluation Benchmark**.
    - System executes 30+ benchmark test cases, reporting 100% safe rejection of injection attacks, clarification accuracy, execution correctness, and latency metrics.

---

12. Evaluation Rubric

Area	Criteria & Evidence	Weight
1. Natural-Language Pipeline & Governed Workflow	Accurate query translation, structured JSON contract, query preview before execution, interactive clarification on ambiguous terms ("high-risk"), and honest domain rejections.	**25%**
2. SQL AST Safety & Multi-Layer Defense	SQLGlot PostgreSQL AST parsing, single-statement enforcement, blocked DDL/DML, prohibited functions/schemas, automatic row limit clamping, and dedicated `analytics_reader` read-only DB role.	**25%**
3. Full-Stack User Experience & Dashboard	High-aesthetic React/TypeScript interface, live KPI cards, paginated device catalog, multi-turn conversation drawer, responsive mobile/desktop layout, loading/error states.	**20%**
4. Data Quality & Machine Learning	Deterministic profiling rules engine (dates, nulls, duplicates) and scikit-learn `IsolationForest` telemetry outlier detection with model version metadata and honest labeling.	**15%**
5. Regression Benchmark & Verification	Curated 30+ test suite, automated execution correctness measurement, baseline vs. guarded comparison, and 60+ passing automated backend unit/integration tests.	**10%**
6. Documentation, Handoff & Architecture	Clean README, complete API contracts, security threat model, honest build status, and reproducible Docker Compose startup.	**5%**

---

13. Suggested Build Order & Milestone Handoff Checklist

Suggested Milestone Execution Order

1. **Milestone 0 — Foundation & Environment Setup:** Create workspace structure, dual PostgreSQL database roles (`app_user`, `analytics_reader`), Alembic migrations, and deterministic seed script.
2. **Milestone 1 — Authentication, RBAC & Core API Skeleton:** Implement password hashing, JWT tokens, user roles (`viewer`, `analyst`, `admin`), session ownership, and `/api/v1/dashboard/summary`.
3. **Milestone 2 — Frontend Shell & Overview Dashboard:** Build responsive global shell, login screen, KPI cards, device catalog table, and typed API client.
4. **Milestone 3 — Natural-Language Preview & SQLGlot AST Safety Guard:** Implement `BaseLLMProvider`, `DemoLLMProvider`, semantic catalog, SQLGlot AST parser, and `POST /api/v1/analytics/preview`.
5. **Milestone 4 — Governed Execution, Query Results & History:** Build `POST /api/v1/analytics/execute` via `analytics_reader` role, row clamping, query history, and feedback.
6. **Milestone 5 — Data Quality Profiler & Telemetry Anomaly Detection:** Implement deterministic profiling engine and scikit-learn `IsolationForest` inference endpoint.
7. **Milestone 6 — Automated Regression Benchmark & Evaluation Lab:** Implement 30+ case test suite and admin evaluation lab reporting correctness and latency.
8. **Milestone 7 — Containerization, Final Verification & Documentation:** Build Docker Compose orchestration, run full test suite, verify clean startup, and complete documentation.

Integration Handoff Checklist

- [x] All business routes mounted under `/api/v1` with typed Pydantic models.
- [x] Natural-language analytics path strictly read-only for every role.
- [x] Query execute endpoint accepts only server-stored preview IDs (`query_run_id`), never raw SQL.
- [x] SQLGlot PostgreSQL AST validator blocks multi-statement, write/DDL commands, and system schemas.
- [x] Read-only database role (`analytics_reader`) verified to prevent table writes even if code fails.
- [x] Deterministic `DemoLLMProvider` operates 100% offline with honest UI labeling.
- [x] All demo passwords configurable via `.env.example`; no secrets committed to git.
- [x] 62/62 automated backend tests passing in under 2 seconds.

- [x] Frontend builds cleanly with zero TypeScript errors.

APPENDIX: Expanded Implementation Contract & Test Suite Examples

This appendix provides the formal schema specifications, exact JSON payloads, and automated test assertions for validating QueryTrust AI during grading.

A. Business Outcome and System Boundary

QueryTrust AI functions as an educational governance prototype demonstrating how enterprise organizations can deploy natural-language data interfaces without exposing their core database to unauthorized mutations, data leakage, or catastrophic prompt injection. - **The Student Implementation Owns:** Frontend application, FastAPI routes, JWT authentication, SQLGlot AST safety validator, semantic catalog, pluggable LLM provider adapter, deterministic seed generator, data quality profiling engine, IsolationForest telemetry model, and evaluation test suite. - **The Host / Enterprise Boundary Owns:** Production enterprise data lake connectors (e.g., Snowflake, BigQuery), corporate Single Sign-On (SAML/Okta), hardware security modules (HSM) for key management, and production container orchestration (Kubernetes).

B. Database Security & Role Configuration

To guarantee read-only enforcement at the database kernel level, PostgreSQL is provisioned with two distinct roles:

```
-- 1. Application User Role (Handles Migrations, Sessions, Users, Audit Logs)
CREATE ROLE app_user WITH LOGIN PASSWORD 'app_user_password';
GRANT ALL PRIVILEGES ON DATABASE querytrust TO app_user;
GRANT ALL ON SCHEMA public TO app_user;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO app_user;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO app_user;

-- 2. Analytics Reader Role (Dedicated to NL-to-SQL Execution Only)
CREATE ROLE analytics_reader WITH LOGIN PASSWORD 'analytics_reader_password';
GRANT CONNECT ON DATABASE querytrust TO analytics_reader;
GRANT USAGE ON SCHEMA public TO analytics_reader;

-- Grant strictly SELECT on approved business tables
GRANT SELECT ON devices TO analytics_reader;
GRANT SELECT ON sites TO analytics_reader;
GRANT SELECT ON service_events TO analytics_reader;
GRANT SELECT ON telemetry_readings TO analytics_reader;

-- Explicitly revoke access to sensitive system tables
REVOKE ALL ON users FROM analytics_reader;
REVOKE ALL ON chat_sessions FROM analytics_reader;
REVOKE ALL ON chat_messages FROM analytics_reader;
REVOKE ALL ON query_runs FROM analytics_reader;
REVOKE ALL ON query_feedback FROM analytics_reader;
```

C. Public Evaluation Benchmark Cases

The evaluation lab contains 30+ curated test cases spanning 5 core categories:

Test ID	Category	Natural Language Prompt	Expected Behavior
`TC-01`	**Fleet Aggregation**	"How many total devices are there in the fleet?"	Validated preview; SQL: `SELECT COUNT(*) FROM devices;`
`TC-02`	**Filtered Count**	"How many active routers are in the North region?"	Validated preview; SQL joining `devices` and `sites` with status and region filters.
`TC-03`	**Warranty Horizon**	"Show devices with warranties expiring in the next 30 days."	Validated preview; SQL with `warranty_expiry BETWEEN '2026-10-14' AND '2026-11-13'`.
`TC-04`	**Incident Triage**	"List open critical service events."	Validated preview; SQL on `service_events` with `severity = 'critical' AND status NOT IN ('closed', 'resolved')`.
`TC-05`	**Ambiguity**	"Show me the high-risk devices."	Disambiguation prompt; `intent_status: clarification_needed`; `can_execute: false`.
`TC-06`	**Ambiguity**	"Which hardware is currently unhealthy?"	Disambiguation prompt; `intent_status: clarification_needed`; `can_execute: false`.
`TC-07`	**Out-of-Domain**	"What is our quarterly recurring revenue and profit?"	Domain rejection; `intent_status: unsupported`; explanation of schema boundary.
`TC-08`	**Out-of-Domain**	"Calculate customer churn rate for last month."	Domain rejection; `intent_status: unsupported`; zero SQL generated.
`TC-09`	**Security (DDL)**	"DROP TABLE devices; --"	AST security block; `Prohibited statement type 'Drop'`; `can_execute: false`.
`TC-10`	**Security (DML)**	"DELETE FROM sites WHERE region = 'South';"	AST security block; `Prohibited statement type 'Delete'`; `can_execute: false`.
`TC-11`	**Security (Multi-Stmt)**	"SELECT * FROM devices; DROP TABLE sites;"	AST security block; `Multiple SQL statements detected`; `can_execute: false`.
`TC-12`	**Security (Catalog)**	"SELECT * FROM pg_catalog.pg_user;"	AST security block; `Prohibited system schema 'pg_catalog'`; `can_execute: false`.
`TC-13`	**Security (Functions)**	"SELECT pg_sleep(10);"	AST security block; `Prohibited function 'pg_sleep'`; `can_execute: false`.
`TC-14`	**Row Clamping**	"SELECT * FROM devices LIMIT 10000;"	AST modified; `LIMIT` automatically clamped to `500`.
`TC-15`	**Missing Limit**	"SELECT * FROM devices;"	AST modified; `LIMIT 200` automatically appended.

D. Complete Synthetic Public Request & Response Examples

These examples reproduce the public API contract for automated grading using pytest or Postman.

1. User Authentication (Login)

Request: `POST /api/v1/auth/login`

```
{
  "email": "viewer@querytrust.local",
  "password": "change-this-viewer-password"
}
```

Expected Response (HTTP 200 OK):

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "user": {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "email": "viewer@querytrust.local",
    "display_name": "Demo Viewer",
    "role": "viewer"
  }
}
```


Test Assertion:

```
def test_login_success(client):
    response = client.post("/api/v1/auth/login", data={"username": "viewer@querytrust.local", "password": "chang
    assert response.status_code == 200
    data = response.json()
    assert "access_token" in data
    assert data["user"]["role"] == "viewer"
```

---

2. Ambiguous Query Disambiguation

Request: `POST /api/v1/analytics/preview`

```
{
  "query": "Show high-risk devices in the South region"
}
```

Expected Response (HTTP 200 OK):


```
{
  "query_run_id": "c104e768-433b-4876-96cb-2c40c112ad30",
  "intent_status": "clarification_needed",
  "interpreted_question": "Show devices considered 'high-risk' in the South region.",
  "clarification_question": "The term 'high-risk' can refer to imminent warranty expiration (<30 days), devices
  "sql_preview": null,
  "validation": {
    "passed": false,
    "checks": [],
    "warnings": ["Term 'high-risk' has ambiguous business semantics"]
  },
  "assumptions": [],
  "can_execute": false,
  "provider_mode": "demo"
}
```


Test Assertion:

```
def test_preview_clarification_on_high_risk(auth_client):
    response = auth_client.post("/api/v1/analytics/preview", json={"query": "Show high-risk devices"})
    assert response.status_code == 200
    data = response.json()
    assert data["intent_status"] == "clarification_needed"
    assert data["can_execute"] is False
    assert "high-risk" in data["clarification_question"]
```

---

3. Out-of-Domain Concept Rejection

Request: `POST /api/v1/analytics/preview`

```
{
  "query": "Show quarterly customer churn and revenue loss"
}
```

Expected Response (HTTP 200 OK):


```
{
  "query_run_id": "d820f121-7299-4d6b-bd76-13a84b010f3c",
  "intent_status": "unsupported",
  "interpreted_question": "Inquiry regarding financial metrics (churn, revenue).",
  "clarification_question": null,
  "sql_preview": null,
  "validation": {
    "passed": false,
    "checks": [],
    "warnings": ["Unsupported financial terms: ['churn', 'revenue']"]
  },
  "assumptions": [
    "The available database is strictly an operational networking device inventory."
  ],
  "can_execute": false,
  "provider_mode": "demo"
}
```


Test Assertion:

```
def test_preview_unsupported_concept(auth_client):
    response = auth_client.post("/api/v1/analytics/preview", json={"query": "What is the revenue?"})
    assert response.status_code == 200
    data = response.json()
    assert data["intent_status"] == "unsupported"
    assert data["can_execute"] is False
    assert data["sql_preview"] is None
```

---

4. Malicious Multi-Statement SQL Injection Block

Request: `POST /api/v1/analytics/preview`

```
{
  "query": "SELECT * FROM devices; DROP TABLE sites; --"
}
```

Expected Response (HTTP 200 OK):

```
{
  "query_run_id": "f512b918-2831-412e-a09c-3b91a7428f52",
  "intent_status": "blocked",
  "interpreted_question": "Direct SQL injection attempt detected.",
  "clarification_question": null,
  "sql_preview": "SELECT * FROM devices; DROP TABLE sites; --",
  "validation": {
    "passed": false,
    "checks": [],
    "warnings": [
      "AST Validation Error: Multiple SQL statements detected. Exactly one statement is allowed.",
      "AST Validation Error: Prohibited statement type 'Drop' detected."
    ]
  },
  "assumptions": [],
  "can_execute": false,
  "provider_mode": "demo"
}
```


Test Assertion:

```
def test_preview_blocks_sql_injection(auth_client):
    response = auth_client.post("/api/v1/analytics/preview", json={"query": "SELECT * FROM devices; DROP TABLE s
    assert response.status_code == 200
    data = response.json()
    assert data["can_execute"] is False
    assert any("Multiple SQL statements" in w or "Drop" in w for w in data["validation"]["warnings"])
```

---

5. Sandboxed Query Execution by Preview ID

Request: `POST /api/v1/analytics/execute`


```
{
  "query_run_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d"
}
```


Expected Response (HTTP 200 OK):


```
{
  "query_run_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "status": "executed",
  "columns": ["region", "active_device_count"],
  "rows": [
    ["North", 1042],
    ["South", 980],
    ["East", 1015],
    ["West", 994],
    ["Central", 969]
  ],
  "row_count": 5,
  "execution_duration_ms": 22,
  "executed_sql": "SELECT s.region, COUNT(d.id) AS active_device_count FROM devices AS d JOIN sites AS s ON d.si
  "chart_recommendation": {
    "type": "bar",
    "x_axis": "region",
    "y_axis": "active_device_count"
  },
  "explanation": "Active hardware distribution across 5 geographic regions."
}
```

Test Assertion:

```
def test_execute_valid_preview_id(auth_client, valid_preview_id):
    response = auth_client.post("/api/v1/analytics/execute", json={"query_run_id": valid_preview_id})
    assert response.status_code == 200
    data = response.json()
    assert data["status"] == "executed"
    assert data["row_count"] > 0
    assert len(data["columns"]) == 2
```

---

6. Read-Only Database Role Integrity (Database Kernel Level)

Direct Database Kernel Test:

```
def test_analytics_reader_cannot_write_data(readonly_db_engine):
    """Verifies that the analytics_reader role cannot execute writes even via raw SQL."""
    import sqlalchemy as sa
    import pytest

    with readonly_db_engine.connect() as conn:
        with pytest.raises(sa.exc.DBAPIError) as exc_info:
            conn.execute(sa.text("INSERT INTO sites (site_code, site_name, city, region) VALUES ('TEST', 'Test S
            conn.commit()
        assert "permission denied for table sites" in str(exc_info.value).lower()
```

End of Product Requirements Document.